МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-13

Выполнил:

Д. В. Шибун

студент группы ПО-13

Проверил:

А. Д. Кулик

13.02.2026

Брест 2026

Цель работы:
приобрести практические навыки разработки API и баз данных

Общее задание:
1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Задание: 22) База данных Центр занятости

База данных:

Структура проекта:

```
src/
│
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── crud.py
│   └── routers/
│       ├── citizens.py
│       ├── employers.py
│       ├── vacancies.py
│       ├── applications.py
│       └── skills.py
│
├── requirements.txt
└── .env
```

Код программы:

config.py — загрузка переменных окружения

```
from dotenv import load_dotenv
import os

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")
```

database.py — подключение к PostgreSQL

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from .config import DATABASE_URL

if not DATABASE_URL:
    raise ValueError("DATABASE_URL is not set")

engine = create_engine(DATABASE_URL, echo=True)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

models.py — модели SQLAlchemy

```
from sqlalchemy import Column, Integer, String, Date, ForeignKey
from sqlalchemy.orm import relationship
from .database import Base

class Citizen(Base):
```

```
    __tablename__ = "citizens"

    id = Column(Integer, primary_key=True)
    full_name = Column(String, nullable=False)
    birth_date = Column(Date)
    email = Column(String, unique=True)
    phone = Column(String)

    applications = relationship("Application", back_populates="citizen")


class Employer(Base):
    __tablename__ = "employers"

    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    address = Column(String)

    vacancies = relationship("Vacancy", back_populates="employer")
```

schemas.py — Pydantic-схемы

```
from pydantic import BaseModel, EmailStr
from datetime import date

class CitizenBase(BaseModel):
    full_name: str
    birth_date: date
    email: EmailStr
    phone: str

class CitizenCreate(CitizenBase):
    pass

class CitizenOut(CitizenBase):
    id: int

    class Config:
        from_attributes = True
```

crud.py — операции с БД

```
from sqlalchemy.orm import Session
from . import models, schemas

def get_citizens(db: Session):
    return db.query(models.Citizen).all()

def create_citizen(db: Session, citizen: schemas.CitizenCreate):
    db_citizen = models.Citizen(**citizen.dict())
    db.add(db_citizen)
    db.commit()
    db.refresh(db_citizen)
    return db_citizen
```

routers/citizens.py — маршруты FastAPI

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from ..database import get_db
from .. import crud, schemas

router = APIRouter(prefix="/citizens", tags=["Citizens"])

@router.get("/", response_model=list[schemas.CitizenOut])
def read_citizens(db: Session = Depends(get_db)):
    return crud.get_citizens(db)

@router.post("/", response_model=schemas.CitizenOut)
def create_citizen(citizen: schemas.CitizenCreate, db: Session = Depends(get_db)):
    return crud.create_citizen(db, citizen)
```

main.py — запуск приложения

```
from fastapi import FastAPI
from .routers import citizens, employers, vacancies, applications, skills

app = FastAPI(title="Employment Center API")

app.include_router(citizens.router)
app.include_router(employers.router)
app.include_router(vacancies.router)
app.include_router(applications.router)
app.include_router(skills.router)
```

Спецификация ввода

Введите команду запуска сервера: <строка>

Пример

Введите команду запуска сервера: uvicorn app.main:app --reload

Спецификация вывода

Запускаем сервер FastAPI...

Команда: <команда_запуска>
Сервер успешно запущен!

Документация доступна по адресу:
<ссылка_на_docs>

Доступные эндпойнты:
- /citizens
- /employers
- /vacancies
- /applications
- /skills

Готово к работе.

Пример

Запускаем сервер FastAPI...

Команда: uvicorn app.main:app --reload
Сервер успешно запущен!

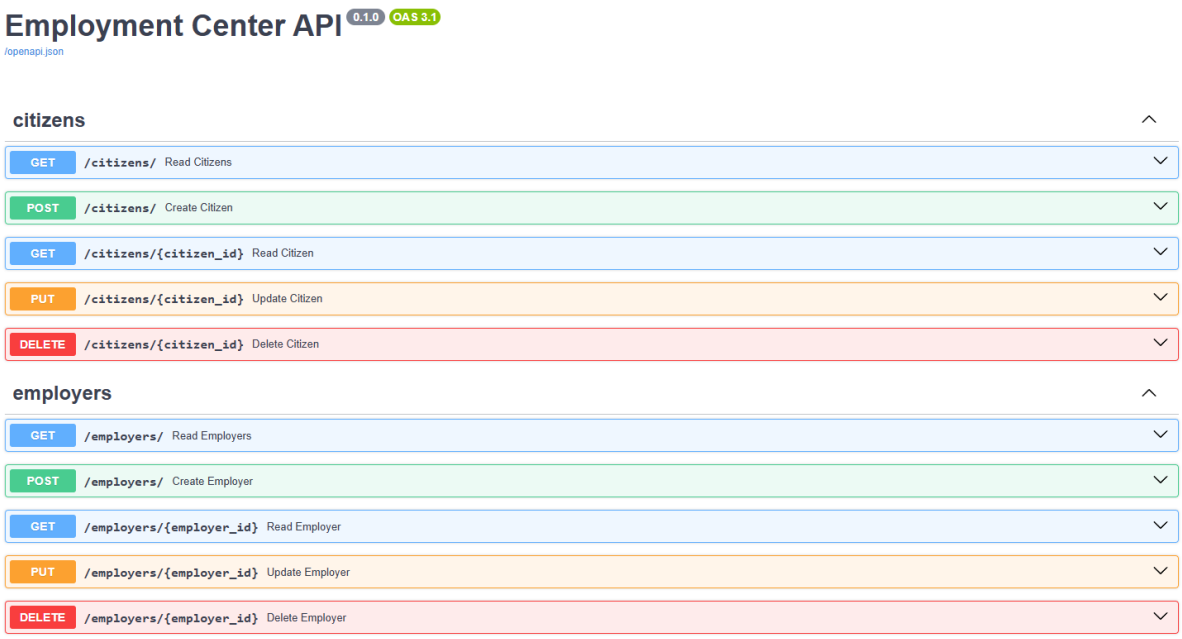
Документация доступна по адресу:
http://127.0.0.1:8000/docs

Доступные эндпойнты:
1. /citizens — работа с соискателями
2. /employers — работа с работодателями
3. /vacancies — работа с вакансиями
4. /applications — работа с откликами
5. /skills — работа с навыками

API готово к использованию.

Рисунки с результатами работы программы

```
(venv) PS D:\Python\spp_po13\reports\Shibun\5\src> uvicorn app.main:app --reload
INFO:     Will watch for changes in these directories: ['D:\\Python\\spp_po13\\reports\\Shibun\\5\\src']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [22956] using StatReload
D:\Python\spp_po13\reports\Shibun\5\src\venv\lib\site-packages\pydantic\_internal\_config.py:383: UserWarning: Valid config keys have changed in V2:
* 'orm_mode' has been renamed to 'from_attributes'
  warnings.warn(message, UserWarning)
INFO:     Started server process [10844]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```



Вывод: В ходе выполнения работы был разработан и развернут серверный веб-приложение на основе FastAPI, обеспечивающее взаимодействие с реляционной базой данных PostgreSQL через ORM-библиотеку SQLAlchemy.